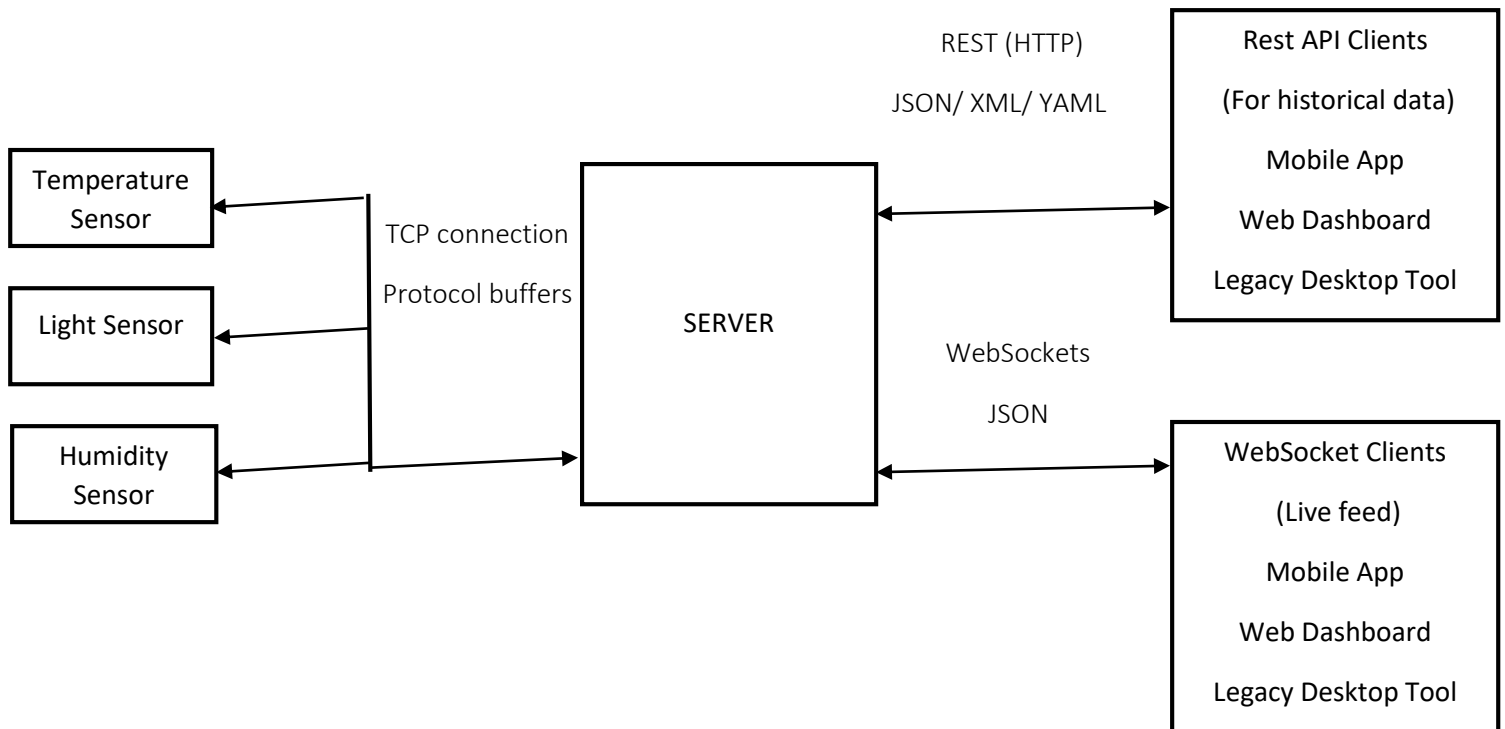
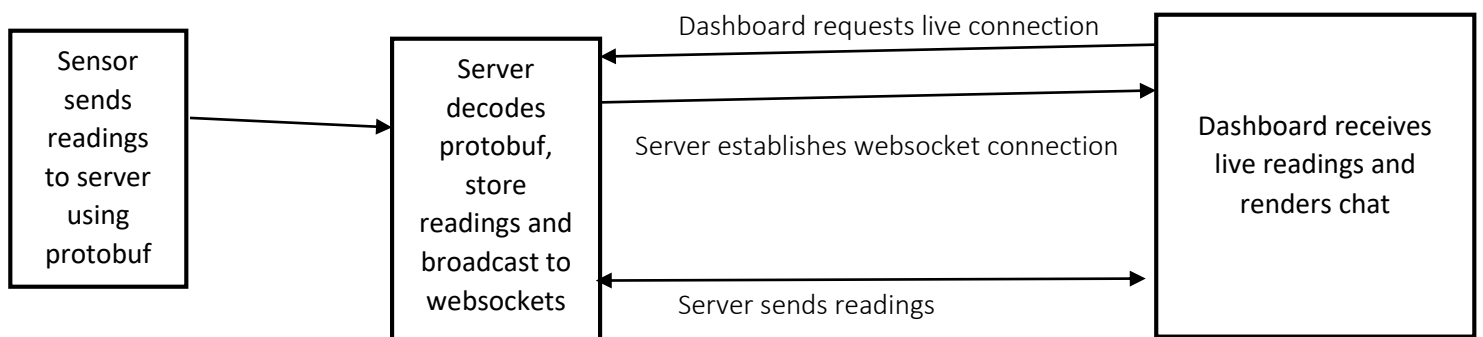


SYSTEM ARCHITECTURE DIAGRAM



SEQUENCE DIAGRAM



DATA MODEL

The prototype stores data in memory using Python data structures rather than a database to minimize system complexities.

Sensor

An object representing a greenhouse sensor device.

FIELD	DATA TYPE	DESCRIPTION
Sensor_id	string	Unique sensor identifier

name	string	Human-readable sensor name
Sensor_type	string	Either temperature , humidity , soil moisture or light sensor
location	String	Greenhouse location
Reporting_interval	integer	Reporting frequency in seconds
active	boolean	Whether sensor is functioning

Reading

A telemetry reading received from a sensor.

FIELD	DATA TYPE	DESCRIPTION
Sensor_id	string	Unique sensor identifier
temperature	float	Temperature value
humidity	float	Humidity value
Soil_moisture	float	Soil moisture percentage
light_level	float	Light intensity
timestamp	integer	timestamp

ClientSession

Used for REST API session tracking through cookies.

FIELD	DATA TYPE	DESCRIPTION
Session_token	string	Unique client session ID
Client_type	string	Mobile, dashboard, desktop
Connected_at	integer	Session creation timestamp
Last_seen	integer	Last activity timestamp

The full `.proto` file

```
syntax = "proto3";

package greenhouse.telemetry;

message Sensor {
    string sensor_uid = 1;
    string name = 2;
    string sensor_type = 3;
    string location = 4;
    int32 reporting_interval = 5;
    bool active = 6;
}
```

```

message Reading {
    string sensor_uid = 1;
    double temperature = 2;
    double humidity = 3;
    double soil_moisture = 4;
    double light_level = 5;
    int64 timestamp = 6;
}

message SensorRegistration {
    Sensor sensor = 1;
}

message TelemetryMessage {
    oneof payload {
        Reading reading = 1;
        SensorRegistration registration = 2;
    }
}

```

REST ENDPOINT SPECIFICATION

Assuming the base URL is <http://localhost:8000>

List Sensors

PROPERTY	DESCRIPTION
Endpoint	/sensors
Method	GET
Request Body	None
Reponse Body	List of sensor objects
Success Status	200 OK
Error Status	406 Not Acceptable

Example Request

GET /sensors HTTP/1.1

Accept: application/json

Example JSON Response

```
[
  {
    "sensor_uid": "temp_sensor_1",
    "name": "Temperature Sensor 1",
    "sensor_type": "temperature",
    "location": "Greenhouse A",
    "reporting_interval": 5,
    "active": true
  }
]
```

Get Historical Readings

PROPERTY	DESCRIPTION
Endpoint	/sensors/{id}/readings
Method	GET
Request Body	None
Reponse Body	List of readings
Success Status	200 OK
Error Status	404 Not Found, 400 Bad Request
Query Parameters	from, to

Example Request

GET /sensors/temp_sensor_1/readings?from=2026-05-01&to=2026-05-02 HTTP/1.1

Accept: application/json

Example JSON Response

```
[
  {
    "sensor_uid": "temp_sensor_1",
    "temperature": 27.4,
    "humidity": 60.1,
    "timestamp": 1778320000
  },

```

```
{
  "sensor_uid": "temp_sensor_1",
  "temperature": 28.0,
  "humidity": 59.5,
  "timestamp": 1778320300
}
```

```
]
```

Register New Sensor

PROPERTY	DESCRIPTION
Endpoint	/sensors
Method	POST
Request Body	Sensor object
Reponse Body	Created sensor
Success Status	201 Created
Error Status	400 Bad Request, 409 Conflict

Example Request

POST /sensors HTTP/1.1

Content-Type: application/json

Example Request Body

```
{
  "sensor_uid": "humidity_sensor_2",
  "name": "Humidity Sensor 2",
  "sensor_type": "humidity",
  "location": "Greenhouse B",
  "reporting_interval": 10,
  "active": true
}
```

Example Response

```
{
```

```
"message": "Sensor registered successfully",  
"sensor_uid": "humidity_sensor_2"  
}
```

Delete Sensor

PROPERTY	DESCRIPTION
Endpoint	/sensors/{id}
Method	DELETE
Request Body	None
Reponse Body	Confirmation message
Success Status	200 OK
Error Status	404 Not Found

Example Request

DELETE /sensors/temp_sensor_1 HTTP/1.1

Example Response

```
{  
  "message": "Sensor removed successfully"  
}
```

YAML configuration schema

Configuration Structure

FIELD	DATA TYPE	DESCRIPTION
Sensor_id	string	Unique sensor identifier
name	string	Human readable sensor name
Sensor_type	String	Type of sensor
location	String	Greenhouse location
reporting-interval	Integer	Time between reports in seconds
active	Boolean	Whether server is functioning or not
Server.host	String	Server IP/domain
Server.port	Integer	Server TCP port
measurements	list	Types of measurements produced

The YAML configuration file example

sensor_uid: temp_sensor_1

name: Temperature Sensor 1

sensor_type: temperature

location: Greenhouse A

reporting_interval: 5

active: true

server:

host: 127.0.0.1

port: 8000

measurements:

- temperature
- humidity

CONCURRENCY MODEL

Used Coroutines

Sensor Connection Coroutine

Handles one TCP-connected sensor to;

- accept telemetry data
- read Protobuf messages
- deserialize readings
- store readings in memory
- forward live updates to WebSocket subscribers

Each sensor connection gets its own coroutine instance.

REST API Request Coroutine

Handles HTTP requests asynchronously to;

- process REST endpoints
- query historical readings
- serialize JSON/XML/YAML responses
- manage cookies/session tracking

Multiple API requests can execute concurrently without blocking each other.

WebSocket Connection Coroutine

Maintains one live dashboard connection to;

- perform WebSocket upgrade
- receive subscriptions
- send live telemetry updates
- detect disconnected clients

Each dashboard client gets a coroutine.

Broadcast Coroutine

Distributes live readings to subscribed WebSocket clients to;

- receive new telemetry readings
- identify interested subscribers
- asynchronously push JSON updates

This prevents sensor ingestion from being tightly coupled to dashboard delivery.

Sensor Simulator Coroutine

Simulates sensor behavior asynchronously to;

- load YAML configuration
- generate telemetry values
- serialize Protobuf messages
- periodically transmit readings

.
All the coroutines communicate through shared in-memory structures and asynchronous queues. When a slow client can't keep up, the server can disconnect the client i.e the server closes the WebSocket connection and resources are reclaimed. This prevents one slow dashboard from affecting the entire system.

When a sensor disconnects, the server makes sure the connection coroutine is terminated, the sensor is marked inactive in memory, the TCP socket is closed, dashboard clients may receive a status update and the server continues serving other sensors normally.

If the storage layer fails and a reading cannot be stored an exception is logged, the malformed reading is discarded, the sensor connection remains active if possible and the server continues processing other requests.

When the server is overloaded it can temporarily reject new sensor connections or additional WebSocket clients and use a timeout to disconnect slow or idle clients to quickly free resources

